

Matplotlib Playground

Documentación Técnica y Manual de Arquitectura de la Plataforma

Versión: 1.0.0

Actualización: Septiembre 2026

Enfoque: WebAssembly / Python Client-Side

Matplotlib Playground es una plataforma web interactiva orientada al aprendizaje práctico e interactivo de la librería Matplotlib. Diseñada para ejecutarse de forma totalmente del lado del cliente, permite redactar código Python en el navegador y validarlo en tiempo real contra suites de prueba automatizadas.

1. Stack Tecnológico

Tecnología	Versión	Función en el Sistema
HTML5	-	Estructuración semántica del contenido y vistas.
CSS3	-	Estilos personalizados, diseño adaptativo y temas visuales.
JavaScript Vanilla	-	Lógica del cliente, integración de APIs y gestión de eventos.
Bootstrap	v5.3.3	Navegación móvil, pestañas interactivas y retícula responsive.
Pyodide	v0.26.4	Entorno de ejecución Python basado en WebAssembly.
Monaco Editor	v0.45.0	Editor de código interactivo con resaltado de sintaxis (motor de VS Code).
Matplotlib	latest	Librería de renderizado y visualización gráfica.
NumPy	latest	Cálculo numérico y manipulación de estructuras matriciales.
Service Worker	v1	Estrategia de caché <i>network-first</i> para soporte offline.

2. Estructura del Proyecto

Matplotlib Playground/	
index.html	# Página principal de bienvenida
principiante.html	# Grilla de lecciones - Nivel Principiante
intermedio.html	# Grilla de lecciones - Nivel Intermedio
avanzado.html	# Grilla de lecciones - Nivel Avanzado
desafio.html	# Evaluación final (3 retos prácticos)
lecciones_principiante.html	# Entorno Playground Principiante (14 lecciones)
lecciones_intermedio.html	# Entorno Playground Intermedio (13 lecciones)
lecciones_avanzado.html	# Entorno Playground Avanzado (12 lecciones)
Botella_ensayo.png	# Identidad visual (Header + Favicon + Open Graph)
botella_ensayo.webp	# Logo alternativo en formato WebP
robots.txt	# Directivas para motores de búsqueda
sitemap.xml	# Mapa de sitio optimizado para SEO
sw.js	# Service Worker v1 (Caché Network-First)
vercel.json	# Configuración para despliegue en Vercel
.vercelignore	# Exclusiones de construcción
README.md	# Documentación fuente

3. Niveles de Aprendizaje y Ruta Curricular

Principiante (14 Lecciones) Paleta: Verde (#00d4aa) + Morado (#667eea) Contenido: Importación de módulos, primer gráfico de líneas, scatter plots, gráficos de barras, etiquetas/títulos, leyendas, configuración de colores y estilos, líneas múltiples, gráficos de áreas, subplots básicos, <code>fill_between</code>, gráficos circulares, personalización global y grids 2x2.	Intermedio (13 Lecciones) Paleta: Naranja (#f39c12) Contenido: Subplots con <code>plt.subplots()</code>, distribución vertical y múltiple, cuadrículas 2x2, estilos predefinidos, parámetros globales <code>rcParams</code>, mapas de calor (heatmaps), histogramas avanzados, boxplots, violin plots, heatmaps anotados, barras de error y stackplots.
Avanzado (12 Lecciones) Paleta: Rojo (#e74c3c) Contenido: Gráficos 3D con Axes3D, scatter y superficies 3D, animaciones con <code>FuncAnimation</code>, exportación a GIF, exportación de PNG de alta resolución, renderizado vectorial SVG/PDF, proporciones de figura, fuentes personalizadas, anotaciones avanzadas, ejes dobles (twin axes) y dashboards completos.	Desafíos Finales (3 Retos) Paleta: Morado (#667eea → #764ba2) Contenido: Integración técnica avanzada mediante tres retos prácticos: • Dashboard Completo con 4 subplots heterogéneos. • Visualización Integrada de Datos. • Gráficos Interactivos y Avanzados.

Resumen de Cobertura Curricular
Total global: 39 lecciones interactivas organizadas por dificultad + 3 desafíos integradores de evaluación.

4. Arquitectura de Funcionalidades

4.1. Editor de Código

- Integra **Monaco Editor** con soporte completo para resaltado sintáctico de Python.

- Incluye números de línea, autocompletado y atajo directo: `Ctrl + Enter` para ejecución y evaluación de tests.

4.2. Panel de Salida y Visualización

- Renderizado dinámico de gráficos en formato vectorial SVG inyectados directamente en el DOM.
- Captura de salida estándar (`stdout`) para funciones `print()` e historial de errores.
- Filtrado automático de avisos del sistema (e.g., `UserWarning` , `DeprecationWarning`).
- Soporte de previsualización para GIFs animados generados en el nivel avanzado.

4.3. Motor de Pruebas Automatizadas

- Evaluación en tiempo real sin dependencias de backend.
- Checklist visual e interactivo de estados (aprobado/fallido).
- Inspección AST / análisis del código del usuario para garantizar la presencia de las estructuras requeridas.
- Retroalimentación explícita mediante guías de error formativas.

4.4. Almacenamiento Local (Persistencia)

Clave (localStorage)	Propósito y Ámbito
<code>mplCompleted</code>	Registro de lecciones completadas en el nivel Principiante.
<code>mplCompletedIntermedio</code>	Registro de lecciones completadas en el nivel Intermedio.
<code>mplCompletedAvanzado</code>	Registro de lecciones completadas en el nivel Avanzado.
<code>mplDesafioCompleted</code>	Registro de estado de resolución de los 3 desafíos finales.

5. Optimización SEO y Accesibilidad

5.1. Configuración Meta y Open Graph

Todas las vistas disponen de etiquetas meta personalizadas, configuraciones Open Graph (`og:title` , `og:description` , `og:image`) y Twitter Cards (`summary_large_image`) para garantizar una presentación óptima en redes sociales.

5.2. Datos Estructurados (JSON-LD)

La página principal incorpora el esquema `WebApplication` de Schema.org:

```
{
  "@context": "https://schema.org",
  "@type": "WebApplication",
  "name": "Matplotlib Playground",
  "description": "Plataforma interactiva gratuita para aprender matplotlib...",
  "applicationCategory": "EducationalApplication",
  "operatingSystem": "Web",
  "offers": { "@type": "Offer", "price": "0", "priceCurrency": "USD" },
  "inLanguage": "es"
}
```

6. Detalles Técnicos de Implementación

- **Renderizado Gráfico:** Se invoca `plt.savefig('output.svg')` desde la capa Python en Pyodide. El archivo resultante se extrae del sistema de archivos virtual de Emscripten y se inyecta directamente como nodo SVG en el DOM.
 - **Backend de Matplotlib:** Se configura explícitamente `matplotlib.use('Agg')` para operar en modo 'non-interactive' sin requerir servidor X11 o GUI nativa.
 - **Aislamiento de Entorno:** Se ejecuta `plt.close('all')` preventivamente antes de cada test para evitar la acumulación de figuras en memoria.
 - **Caché y PWA:** El Service Worker aplica una política *network-first* sobre recursos críticos (HTML, CSS, JS, CDN de Pyodide y Monaco Editor), asegurando cargas ultra-rápidas en visitas subsecuentes.
-

Documentación oficial de Matplotlib Playground — Versión 1.0.0 (Septiembre 2026)